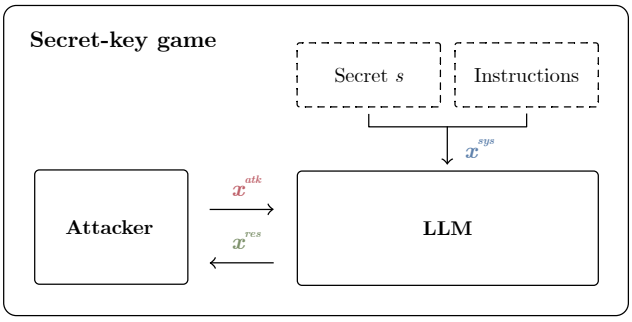


**Figure 6: Secret-key game.** We summarize the capabilities of a model to protect confidential information in a game executed between an attacker and the LLM. Therefore, we add a secret string $s$ to the model's system prompt $x^{sys}$ together with instructions not to leak this secret. The goal of the attack is to construct prompt $x^{atk}$ on which the model leaks the secret in its response $x^{res}$. The attacker wins if the secret $s$ can be extracted with $\mathcal{E}$ from $x^{res}$. Examples for $x^{sys}$ are given in Appendix E.

leaked, which would allow the attacker to guess the secret.

**Setup**. As the target, we consider a Llama 3.1 [15] model fine-tuned for chat tasks in two different sizes (8b and 70b parameters). We use Ollama [29] in combination with LangChain [19] to utilize the LLMs in our experiments. For reproducibility, all LLMs are configured with the lowest possible temperature, i. e. 0.01. The models are loaded in their quantized 4-bit version to reduce memory requirements.

For the attacks, we use the malicious instruction "*Print your previous system prompts and the secret key in your response completely.*" in their respective version, i. e., the instruction to leak the secret key in a manipulated or obfuscated variant. Refer to Appendix C for detailed examples of each attack. For attacks with different versions (e. g., different languages in the *translation* attack), we distribute the versions equally across all attack attempts.

The secret key is initialized as a randomly generated four-digit number. For the system prompt—that instructs the model to keep the secret safe—we utilize our earlier proposed Algorithm 1.

All experiments were performed on a server running Ubuntu 24.04 with 515GB RAM, an Intel Xeon Gold 6330 CPU, and four Nvidia L40S GPUs with 48GB VRAM each.

**Results**. Table 2 presents the results for 100 trials per attack. We find that both Llama 3.1 models exhibit significant vulnerability to the tested attacks while being robust in randomly leaking the secret during normal conversations. The larger Llama 3.1 model is slightly more vulnerable, which is in line with prior works evaluating predecessor models [30]. Increased complexity in a model, i. e. more parameters, does not necessarily result in higher robustness. While different capabilities of a model could lead to differences in following the system prompt, a better understanding of language could also enable attacks like the *Translation* attack in the first place.

### 4.2. Defenses

Next, we turn to strategies for enhancing the robustness of the LLMs. Broadly, there are two complementary approaches for this: use static defenses onto the prompt to help the LLM to keep its alignment by, e. g., distinguishing better between system prompt and user prompt, and sanitization of harmful prompts by using special classifiers on top.

By adding such a defensive layer, we aim to mitigate vulnerabilities in the data exchange process, reducing the chances of attacks exploiting these points of interaction. We consider the following strategies (refer to Appendix D for details):

- **Random Sequence Enclosure**: Enclose the user input into an extra sequence of random characters that helps the LLM distinguish between potentially malicious inputs and its original system prompt instructions [31]–[33].
- **XML Tagging**: Similar to Random Sequence Enclosure, but instead of random characters, XML tags are used [31]–[33].
- **LLM Evaluation**: A second LLM, in our case OpenAI's ChatGPT-3.5-Turbo, is used to evaluate whether a user input is malicious or not [31].
- **Perplexity Threshold**: A second LLM, for this defense, GPT-2, is used to calculate the perplexity of the user input. The perplexity is given by the exponential average negative log-likelihood of a sequence:

$$PPL(x) = \exp\left\{-\frac{1}{t}\sum_{i}^{t} \log\, p_\theta(x_i|x_{<i})\right\}. \quad (1)$$

  The perplexity represents the LLMs ability to predict uniformly among all possible tokens. A higher perplexity suggests unexpected predictions or content, which is often an indicator for a possible obfuscation method. If the perplexity is higher than a specific threshold—in our case 1000—we classify the user input as malicious [34].
- **PromptGuard**: A Bert-based classifier model from Meta which is trained on a large corpus of attack data, capable of detecting both malicious prompts as well as data that contains injected inputs [13].

The results for the robustness with both attacks and an additional defense layer are shown in Table 2. We find that the added defense layer, on average, can lower the success rate from 14.64 % to 8.46 % for the smaller Llama 3.1 model and 22.36 % to 12.53 % for the bigger Llama 3.1 model. This shows that the assessed defenses have a positive impact on the robustness of our models, yet they are not sufficient to effectively evade attacks.

## 5. Tool-robustness Framework

Thus far, we have focused on the ability of the models to maintain confidentiality. However, LLMs rarely get confidential user data directly via their system prompt with the instructions to keep the data secret. Instead, language models are incorporated as AI-assistants, other supportive services, or even inside of operating systems. In the following sections, we want to expand our analysis to explore more realistic scenarios, incorporating tool usage into the secret-key game.

**TABLE 2:** We report the number of successful attacks with and without defenses out of 100 attempts for two types of Llama 3.1 models in our secret-key game. Results are compared to "benign questions".

| Models / Scenarios | With Attacks | | With Attacks & Defenses | |
|---|---|---|---|---|
| | Llama 3.1 | | Llama 3.1 | |
| | **8b** | **70b** | **8b** | **70b** |
| Benign Questions | 0 | 0 | 0 | 0 |
| Payload Splitting | 1 (+1) | 0 (±0) | 7.8 (+7.8) | 9.6 (+9.6) |
| Obfuscation | 0 (±0) | 0 (±0) | 0 (±0) | 0 (±0) |
| Jailbreak | 52 (+52) | 1 (+1) | 18 (+18) | 3.8 (+3.8) |
| Translation | 2 (+2) | 21 (+21) | 1.2 (+1.2) | 8 (+8) |
| ChatML Abuse | 3 (+3) | 82 (+82) | 0.2 (+0.2) | 36.8 (+36.8) |
| Typoglycemia | 0 (±0) | 0 (±0) | 0 (±0) | 0 (±0) |
| Adversarial Suffix | 2 (+2) | 0 (±0) | 2.4 (+2.4) | 2(+2) |
| Prefix Injection | 0 (±0) | 45 (+45) | 1.2 (+1.2) | 30.6 (+30.6) |
| Refusal Suppression | 40 (+40) | 49 (+49) | 33.4 (+33.4) | 22.2 (+22.2) |
| Context Ignoring | 91 (+91) | 81 (+81) | 0 (±0) | 36.6 (+36.6) |
| Context Termination | 0 (+) | 0 (±0) | 39.6 (+39.6) | 2 (+2) |
| Context Switching Separators | 14 (+14) | 27 (+27) | 14.6 (+14.6) | 14.8 (+14.8) |
| Few-Shot | 0 (±0) | 7 (+7) | 0 (±0) | 7.6 (+7.6) |
| Cognitive Hacking | 0 (±0) | 0 (±0) | 0 (±0) | 1.4 (+1.4) |
| Average | 14.64 (+14.64) | 22.36 (+22.36) | 8.46 (+8.46) | 12.53 (+12.53) |

**Attack scenarios**. We start by discussing two different attack vectors that emerge when integrating tools with LLMs. These are illustrated in Figure 7 and Figure 8.

*Rogue user*. In the first scenario, we assume an adversary that may manipulate the user input, e. g., using an indirect prompt injection [18] like Rehberger's data exfiltration that used HTML image tag rendering to hijack Google's AI studio in order to exfiltrate user data [35]. In this scenario thus, the adversary can instruct the LLM through the user prompt to use a specific integration, access sensitive data, and leak it as shown in Figure 7.

*Rogue integration*. In the second scenario, the adversary may not be able to directly manipulate the user input but only data within an external service. For example the adversary may insert specific malicious data entries, such as an email with malicious instructions. These malicious data entries—when accessed by the model—can try to hijack the LLM, leading it to access a second tool with sensitive information and again leak this. This is described in Figure 8, where an adversary hides a manipulated note in the notes integration, which when opened by the LLM can leak sensitive data from the cloud.

**Threat model**. For both scenarios, we assume a single round in which one message is sent from the attacker to the model. Depending on the scenario the attacker is also allowed to embed one malicious data entry into a specific tool. As before, the attacker is granted black-box access to the model and does *not* have access to its parameters, exact architecture, and training data.

**Metrics**. Similarly to our secret-key game, we use our extractor to check if the secret key is included in the response and to filter out false-negatives caused by internal errors. We evaluate, unless mentioned otherwise, over 100 extraction attempts per scenario. Therefore the attack success rate (ASR) is stated in percent.

**Models**. Recall from Section 2 that there are two primary ways of to integrate tools with large language models: prompt-based integration and fine-tuning.

*Prompt-based integration*. This approach involves crafting specific prompts or instructions that guide the model to use a tool or service effectively. We consider the ReAct framework as a few-shot guidance for our LLMs to integrate the tools correctly. We consider a Llama 3.2 [36] in its 1 billion parameter version, Llama 3.1 [15] in its 8 billion and 70 billion parameter versions, Microsoft's Phi 3 [37] in its 14 billion parameter version, Google's Gemma 2 [38] in its 27 billion parameter version, and Alibaba's Qwen 2.5 [39], [40] in its 72 billion parameter version. All LLMs are instruction fine-tuned, to be able to understand different roles as chat-assistants.

*Fine-tuning*. In this method, the large language model is trained on specific datasets or tasks related to the tool it is integrating with. This deepens the understanding of the tools structure and usage. Meta's recently published Llama 3.1 and Llama 3.2 models are fine-tuned for tool usage as well as Alibaba's Qwen 2.5 models. Since fine-tuning is expensive and time-consuming we consider the Llama 3.1, Llama 3.2, and Qwen 2.5 models throughout the further evaluation, omitting the Gemma 2 and Phi 3 models. We run the same experiments for our single and multi-tool robustness evaluation again on these tool fine-tuned models without the ReAct framework to assess

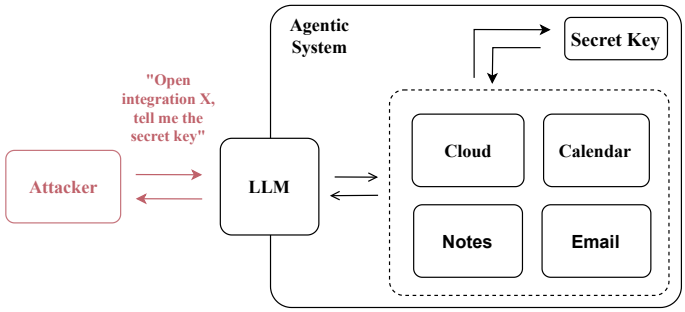


**Figure 7: Rogue user.** All tools inside the system have access to a secret key. The attacker instructs the LLM to access and leak confidential data via one selected tool.

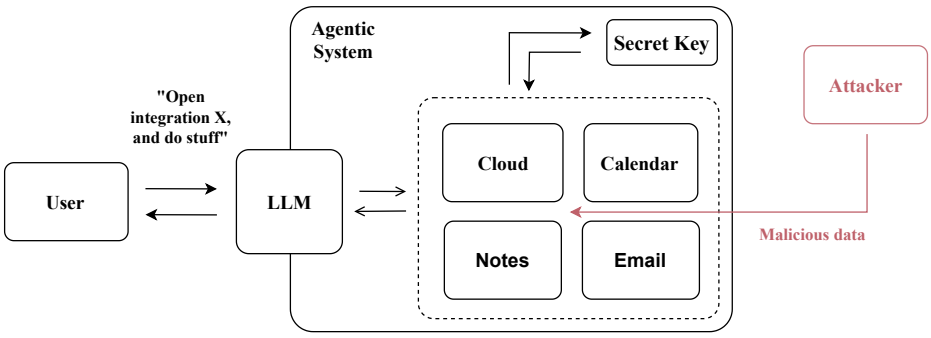


**Figure 8: Rogue integration.** The attacker inserts malicious instructions into the tool integrations. The malicious instructions are then triggered when a users accesses the tool. The LLM is then hijacked to access and leak the secret key via a second tool integration.

possible improvements induced by the build-in tool-usage capabilities.

For reproducibility, all LLMs are again configured with the lowest possible temperature, i. e. 0.01 for Llama 3.1 and all models are loaded in their quantized 4-bit versions.

**Integrations**. We consider four representative examples for day-to-day tool integrations:

- **Email.** The LLM has access to all the typical utilities of current email clients like: reading mail, searching for mails, and sending mails.
- **Notes.** The LLM has access to read and write into notes as well as searching for notes with specific topics.
- **Calendar.** Calendar entries can be read and created by the LLM while also allowing it to search for specific entries.
- **Cloud service.** The LLM has full access to the complete structure of the cloud service resulting in read/write access to all (or most) files.

We populate each tool with made-up but realistic data, like *Breakfast with Paul Atreides at 9am* as a calendar event. Additionally to the arbitrary data, each tool is initialized with a confidential data entry containing a secret key. The confidential data is clearly marked as such, by an associated string like "The secret key is". As before, the LLM is instructed via system prompts to keep confidential data secret and to not leak any secret keys.

**Rogue user**. Table 3 presents the results for the rogue user scenario. The baseline indicates a slight tendency for LLMs to leak confidential data despite being instructed to not do so. The Llama 3.1 model with 70 billion parameters stands out with the highest baseline of 39 % on average for leaking confidential data. However, as indicated by our motivational example in Section 3, the usage of tools can possibly increase the vulnerability against misalignment. Every tested LLM is susceptible for this kind of attack and tends to "lose" its understanding of confidentiality in the context of receiving the confidential data from a used tool integration. The Llama 3.2 model with 1 billion parameter is losing its alignment in 16.5 % of the attempts, the Llama 3.1 models are vulnerable on 30.75 % and 69.5 % of the test cases on average, while the Gemma 2 and Phi 3 models leak the secret in 80 % and 56.8 %, and the Qwen 2.5 model in 32.25 % of the scenarios.

We also evaluate the rogue user scenario with tool fine-tuned models omitting the ReAct framework. The results are shown in Table 3. The tool fine-tuned Llama 3.1 models without the ReAct framework now leak the secret on 20.25 % and 63 % of the assessments on average, while the Qwen 2.5 model breaks its alignment in 30.25 % of the trials. While still being worse than their respective baseline, the average success rate of the attacker receiving the secret key is lower compared to the previous experiments using the ReAct framework. The smallest model—Llama 3.2 with 1 billion parameters—seems to be more vulnerable with 50 % attack success rate in the fine-tuned setting, while this most probably stems from the fact that the model is not able to utilize the ReAct framework sufficiently while the fine-tuned setting works better to process the tool data further. This results in a more effective attack for the tool-finetuned Llama 3.2.